

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: listberkait.c

Di kota penuh anomali Nangor Falls, Dipper Pines sedang pusing tujuh keliling. Mabel Pines baru saja merobek-robek halaman Jurnal Ketiga miliknya! Beruntung, Dipper masih mengingat urutan halaman-halaman tersebut, namun ia butuh struktur penyimpanan yang fleksibel untuk menyusunnya kembali tanpa harus menyiapkan ruang penyimpanan yang kaku.

Melihat hal itu, **Stan** menyarankan Dipper untuk menggunakan struktur data **Linked List (List Berkait)**. Sayangnya, komputer di Mystery Shack sudah sangat tua dan ADT Linked List-nya hilang entah ke mana.

Tugas Anda di sesi Pra-Praktikum ini adalah **Mengimplementasikan seluruh fungsi dan prosedur ADT Linked List linier!** Anda harus mengimplementasikan semua yang ada pada file berikut [listberkait.h](#) ke dalam [listberkait.c](#).

Format Masukan & Keluaran:

Sistem/Driver akan menguji ADT Anda secara otomatis dengan memberikan serangkaian perintah penambahan, penghapusan, dan pencetakan elemen secara berurutan. Jika implementasi pointer Anda meleset sedikit saja, Jurnal Dipper akan hilang ke dimensi lain (baca: *Segmentation Fault*).

Kumpulkan implementasi Anda dalam file [listberkait.c](#)!

Catatan:

- Usahakan untuk mendapat nilai sempurna pada soal ini. Program yang dikerjakan akan digunakan kembali untuk pengerjaan sesi Praktikum nantinya
- Jangan lupa `free()` setiap node yang dihapus agar tidak terjadi *memory leak*
- [boolean.h](#) (jika butuh)

C

 [listberkait.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	9	Accepted	0.00 sec, 1.54 MB
2	9	Accepted	0.00 sec, 1.64 MB
3	9	Accepted	0.00 sec, 1.64 MB
4	9	Accepted	0.00 sec, 1.71 MB
5	9	Accepted	0.00 sec, 1.55 MB
6	9	Accepted	0.00 sec, 1.55 MB
7	9	Accepted	0.00 sec, 1.62 MB
8	9	Accepted	0.00 sec, 1.62 MB
9	9	Accepted	0.00 sec, 1.50 MB
10	9	Accepted	0.00 sec, 1.60 MB
11	10	Accepted	0.00 sec, 1.55 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: anomalinangor.c

Mabel Pines menyadari ada yang aneh dengan fluktuasi energi di sekitar Mystery Shack. Terkadang energi anomali tersebut stabil, namun terkadang terus membesar tanpa henti, yang mana bisa memicu Weirdmageddon skala kecil! Mabel telah mengumpulkan data pembacaan energi anomali tersebut ke dalam sebuah Linked List.

Untuk membantu Mabel menganalisis ancaman, Anda diminta membuat dua fungsi traversal khusus yang akan memeriksa senarai energi tersebut:

1. `boolean isEscalating(List L)`: Mengembalikan `true` jika tingkat energi anomali **terbaca terus meningkat** (strictly increasing) dari awal hingga akhir list. Artinya, elemen selanjutnya harus SELALU lebih besar dari elemen sebelumnya. Jika list kosong atau hanya berisi 1 elemen, kembalikan `true`.
2. `int peakAnomaly(List L)`: Mengembalikan nilai energi anomali terbesar (maksimum) di dalam list. Asumsikan list tidak pernah kosong saat fungsi ini dipanggil.

Berikut file [anomalinangor.h](#) untuk diimplementasikan

Format Masukan:

Sistem/Driver akan otomatis memanggil fungsi yang Anda buat dan mencetak hasilnya.

Implementasikan kedua fungsi tersebut di dalam file `anomalinangor.c`!

Catatan Penting Efisiensi:

- Gunakan struktur dan primitif yang telah Anda buat pada soal sebelumnya di `listlinier.h`.
- Anda WAJIB menelusuri (traverse) list tersebut menggunakan pergeseran pointer (contoh: `p = NEXT(p)`).
- **DILARANG** melakukan iterasi menggunakan kombinasi `for` loop dan fungsi `getElmt()` maupun `length()`!

Penggunaan `getElmt` di dalam loop sangat tidak efisien untuk Linked List dan akan membuat program Anda terkena **Time Limit Exceeded (TLE)** pada test case berukuran besar.

C

 [anomalinangor.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.56 MB
2	10	Accepted	0.00 sec, 1.58 MB
3	10	Accepted	0.00 sec, 1.55 MB
4	10	Accepted	0.00 sec, 1.67 MB
5	10	Accepted	0.00 sec, 1.62 MB
6	10	Accepted	0.00 sec, 1.50 MB
7	10	Accepted	0.00 sec, 1.64 MB
8	10	Accepted	0.00 sec, 1.64 MB
9	10	Accepted	0.00 sec, 1.64 MB
10	10	Accepted	0.01 sec, 3.17 MB